
Roadmap — Phase 2

RAG · FAISS · LLM · React Mobile

Projet Chatbot SUP'PTIC

Planification detaillee sur 3 semaines (15 jours ouvres) —
Pipeline RAG, enrichissement des donnees par binomes circulaires,
interface React mobile et gestion de la base de donnees

Version : 2.0.0
Date : Avril 2026
Duree : 3 semaines, 15 jours ouvres
Equipes : Backend (3) · BD (1) · Data (4) · Frontend (2)

Table des matières

1	Contexte et objectifs de la Phase 2	3
1.1	État des lieux — Phase 1 livrée	3
1.2	Limites identifiées justifiant la Phase 2	3
1.3	Objectifs de la Phase 2	3
1.4	Répartition des équipes	3
2	Vue d'ensemble — Planning 3 semaines	4
3	Semaine 1 : Fondations RAG et Setup React	5
3.1	Objectifs de la semaine	5
3.2	Jour 1 — Lundi : Setup et environnements	5
3.2.1	Équipe Backend (3 membres)	5
3.2.2	Équipe Frontend (2 membres)	5
3.2.3	Équipe Structuration Données (4 membres)	5
3.2.4	Équipe Base de Données (1 membre)	5
3.3	Jour 2 — Mardi : MiniLM et Embeddings	6
3.3.1	Équipe Backend	6
3.3.2	Équipe Structuration Données	6
3.3.3	Équipe Frontend	6
3.3.4	Fin de journée	6
3.4	Jour 3 — Mercredi : FAISS et Indexation	7
3.4.1	Équipe Backend	7
3.4.2	Équipe Structuration Données	7
3.4.3	Équipe Frontend	7
3.4.4	Fin de journée	7
3.5	Jour 4 — Jeudi : Pipeline RAG (sans LLM)	8
3.5.1	Équipe Backend	8
3.5.2	Équipe Frontend	8
3.5.3	Équipe Structuration Données	8
3.5.4	Fin de journée	8
3.6	Jour 5 — Vendredi : Intégration FAISS + Django	9
3.6.1	Équipe Backend	9
3.6.2	Équipe Frontend	9
3.6.3	Équipe Structuration Données	9
3.6.4	Fin de journée — Bilan S1	9
3.7	Jour 6 — Samedi : Rattrapage et documentation S1	10
3.7.1	Tous ensemble	10
4	Semaine 2 : LLM (Phi-3) et Composants React avancés	11
4.1	Objectifs de la semaine	11
4.2	Jour 7 — Lundi : Ollama et LLM Client	11
4.2.1	Équipe Backend	11
4.2.2	Équipe Frontend	11
4.2.3	Équipe Structuration Données	11
4.2.4	Fin de journée	11
4.3	Jour 8 — Mardi : Prompt Builder et RAG complet	12
4.3.1	Équipe Backend	12
4.3.2	Équipe Frontend	12
4.3.3	Fin de journée	12
4.4	Jour 9 — Mercredi : Fallback TF-IDF	13

4.4.1	Équipe Backend	13
4.4.2	Équipe Frontend	13
4.4.3	Équipe Structuration Données	13
4.4.4	Fin de journée	13
4.5	Jour 10 — Jeudi : Feedback et endpoints complémentaires	14
4.5.1	Équipe Backend	14
4.5.2	Équipe Frontend	14
4.5.3	Équipe Base de Données	14
4.5.4	Fin de journée	14
4.6	Jour 11 — Vendredi : Optimisations et tests	15
4.6.1	Équipe Backend	15
4.6.2	Équipe Frontend	15
4.6.3	Équipe Structuration Données	15
4.6.4	Fin de journée	15
4.7	Jour 12 — Samedi : Documentation technique S2	16
4.7.1	Tous ensemble	16
5	Semaine 3 : Intégration finale et livraison	17
5.1	Objectifs de la semaine	17
5.2	Jour 13 — Lundi : Finalisation React	17
5.2.1	Équipe Frontend	17
5.2.2	Équipe Backend	17
5.2.3	Fin de journée	17
5.3	Jour 14 — Mardi : Tests d'intégration	18
5.3.1	Tous ensemble — Session tests croisés	18
5.3.2	Matrice de tests	18
5.3.3	Fin de journée	18
5.4	Jour 15 — Mercredi : Correction et polissage	19
5.4.1	Par équipe	19
5.4.2	Documentation	19
5.4.3	Fin de journée	19
5.5	Jour 16 — Jeudi : Déploiement et répétition démo	20
5.5.1	Déploiement	20
5.5.2	Répétition démo	20
5.5.3	Fin de journée	20
5.6	Jour 17 — Vendredi : Démo officielle	21
5.6.1	Matin — Derniers réglages	21
5.6.2	Après-midi — Démo (14h-17h)	21
5.6.3	Fin de journée	21
5.7	Jour 18 — Samedi : Rétrospective et documentation finale	22
5.7.1	Rétrospective Phase 2	22
5.7.2	Livrables finaux	22
6	Récapitulatif des livrables — Phase 2	23
6.1	Code et configuration	23
6.2	Documentation	23
6.3	Démo	23
7	Indicateurs de succès	24
8	Risques et plans de contingence	25

1 Contexte et objectifs de la Phase 2

1.1 Bilan Phase 1

Ce qui a été livré en Phase 1

- ✓ Recherche TF-IDF + similarité cosinus fonctionnelle
- ✓ Base de données Django avec les Q/R existantes
- ✓ API REST Django (5 endpoints documentés)
- ✓ Interface PWA (HTML/CSS/JS)
- ✓ Système de feedback et statistiques
- ✗ Collecte de données : objectif 1000 Q/R **non atteint**
- ✗ Recherche sémantique : absente (TF-IDF ne comprend pas le sens)
- ✗ Génération de réponses naturelles adaptées au contexte : absente
- ✗ Mémoire conversationnelle : absente

1.2 Limites identifiées justifiant la Phase 2

La Phase 1 repose exclusivement sur TF-IDF qui compare des mots, non du sens. Deux questions synonymes peuvent produire des résultats différents. Le chatbot renvoie des réponses brutes extraites de la base sans adaptation au contexte ni au fil de la conversation. La diversité des formulations réelles des étudiants n'est pas couverte et le dataset reste insuffisant pour assurer une bonne couverture des sujets.

1.3 Objectifs de la Phase 2

1. **Pipeline RAG complet** : MiniLM → FAISS → LLM Phi-3 (Ollama)
2. **Basculer automatique** : RAG si score FAISS ≥ 0.55 , TF-IDF sinon
3. **Interface React mobile** calquée sur la maquette Supone AI (mobile-first)
4. **Dataset enrichi** : ≥ 1050 nouveaux Q/R valides (cible totale : 1500+ en base)
5. **Base de données** cohérente, schéma étendu, synchronisée avec l'index FAISS

1.4 Répartition des équipes

Equipe	Effectif	Branche	Mission
Backend	3	backend	Pipeline RAG, FAISS, Ollama, endpoints, fallback TF-IDF
Base de Données	1	database	Schema, migrations, import JSON, synchronisation FAISS
Data	4	data	Collecte et validation Q/R par binômes circulaires
Frontend	2	frontend	Application React mobile (Supone AI), streaming, UX

2 Processus de collecte des donnees — Equipe Data

2.1 Nouveau processus : binomes circulaires avec validation immediate

L'équipe de 4 membres est divisee en **2 binomes permanents** qui alternent chaque jour entre la collecte et la validation. La regle fondamentale est la suivante :

Regle d'or — synchronisation collecte/validation

Le binome en validation traite **le jour meme** les Q/R produits par l'autre binome **ce meme jour**. Il ne valide jamais sa propre production. Cela garantit que le lendemain, lorsque les roles s'inversent, chaque binome traite une production fraiche qu'il n'a pas lui-meme ecrite — ce qui assure une relecture independante et objective.

Le flux quotidien est : **collecte le matin** → **remise des brouillons a 12h** → **validation l'apres-midi** → **commit + verification du quota le soir**.

Binome	Membres	Jours impairs (J1, J3, J5...)	Jours pairs (J2, J4, J6...)
A	Membre 1 + Membre 2	Collecte (matin)	Validation (apres-midi)
B	Membre 3 + Membre 4	Validation (apres-midi)	Collecte (matin)

Fonctionnement quotidien detaille

- 9h–12h — Binome collecteur** : chaque membre redige **35 Q/R minimum** sur un sous-theme distinct du jour. Total attendu par journee : **70 Q/R bruts**.
- 12h — Remise** : le binome collecteur pousse ses fichiers JSON bruts dans le dossier de travail partage (branche `data`, sous-dossier `brouillons/`).
- 13h–17h — Binome valideur** : lit chaque entree, verifie les criteres (cf. section suivante), corrige la mise en forme JSON, deplace les rejets dans `brouillons/rejets_jX.json` avec une note explicative.
- 17h30 — Commit** : le binome valideur commite le fichier JSON valide dans `data/` avec un message clair : `feat(data): faq_admissions_j1 - 68 Q/R valides`.
- 18h — Script de verification** : lancer `verify_quota.py` pour confirmer que le quota journalier est atteint.

2.2 Criteres de validation

Un Q/R est **valide** s'il respecte tous les criteres suivants :

- La reponse fait au moins 2 phrases completes et apporte une information concrete et verifiable.
- Le champ `exemples` contient au minimum 2 variantes de formulation de la question.
- Le JSON est syntaxiquement correct (aucune erreur de parse avec `json.load()`).
- L'entree n'est pas un doublon d'une entree deja existante (verification manuelle par mot-cle).
- Le champ `sous_theme` correspond bien au theme du jour.
- Le champ `valide` est explicitement mis a `true`.

Un Q/R **rejete** est deplace dans `brouillons/rejets_jX.json` avec une note. Le binome collecteur doit corriger les rejets lors de sa prochaine session de collecte avant d'atteindre le quota du jour.

2.3 Script de verification des quotas

Le script `verify_quota.py` est cree des le Jour 1 et execute chaque soir apres le commit du binome validateur. Il garantit que le calcul de 70 Q/R par jour est bien respecte.

Listing 1 – `verify_quota.py`

```
# verify_quota.py
# Usage : python verify_quota.py --file faq_admissions_j1.json --quota
70

import json, argparse, sys

parser = argparse.ArgumentParser()
parser.add_argument("--file", required=True)
parser.add_argument("--quota", type=int, default=70)
args = parser.parse_args()

with open(args.file, "r", encoding="utf-8") as f:
    data = json.load(f)

total    = len(data)
valides  = sum(1 for d in data if d.get("valide") is True)
rejets   = total - valides

print(f"[verify_quota] Fichier : {args.file}")
print(f"  Total entrees    : {total}")
print(f"  Valides           : {valides}")
print(f"  Rejets            : {rejets}")
print(f"  Quota attendu     : {args.quota}")

if valides < args.quota:
    print(f"  ECHEC : {args.quota - valides} Q/R manquants !")
    sys.exit(1)
else:
    print(f"  OK : quota atteint ({valides} >= {args.quota})")
    sys.exit(0)
```

En cas d'echec (code de sortie 1), le binome collecteur est notifie pour completer sa production avant la fin de journee.

2.4 Format JSON attendu (v2)

Listing 2 – Format JSON v2 — exemple complet

```
{
  "id": "faq_001",
  "categorie": "Admissions",
  "sous_theme": "Dossier d'inscription",
  "question": "Quels documents fournir pour s'inscrire a SUP'PTIC ?",
  "reponse_enrichie": "Le dossier comprend : formulaire d'inscription,
    photocopie du diplome, acte de naissance, photos d'identite.
    Tous les documents doivent etre fournis en 2 exemplaires.",
  "exemples": [
    "Comment s'inscrire a SUP'PTIC ?",
    "Quelles pieces fournir pour l'admission ?",
    "Qu'est-ce qu'il faut pour le dossier d'entree ?"
  ],
  "sources": ["Service des admissions", "brochure_2025.pdf"],
```

```

"date_creation": "2026-04-07",
"version": "2.0",
"valide": true
}

```

Conseils pour les collecteurs

- Chaque membre du binome choisit un **sous-theme distinct** pour éviter les doublons internes.
- Privilégier des questions réellement posées par des étudiants, y compris les formulations orales ou avec des fautes de frappe (à inclure dans **exemples**).
- La **reponse_enrichie** doit être autonome : un étudiant ne doit pas avoir besoin de chercher ailleurs.
- En cas d'incertitude sur une réponse, mettre **"valide": false** et noter la source à vérifier.

2.5 Sous-thèmes et quotas sur 3 semaines

Semaine	Sous-thèmes cibles	Jours	Quota
S1	Admissions et inscriptions, Frais de scolarité, Logement	J1–J5	350 Q/R
S2	Emploi du temps et cours, Examens et notes, Filières	J6–J10	420 Q/R
S3	Services administratifs, Vie étudiante, Ajustements finaux	J11–J14	280 Q/R
Total cible (nouveaux Q/R)			≥ 1050 Q/R

3 Vue d'ensemble — Planning 3 semaines

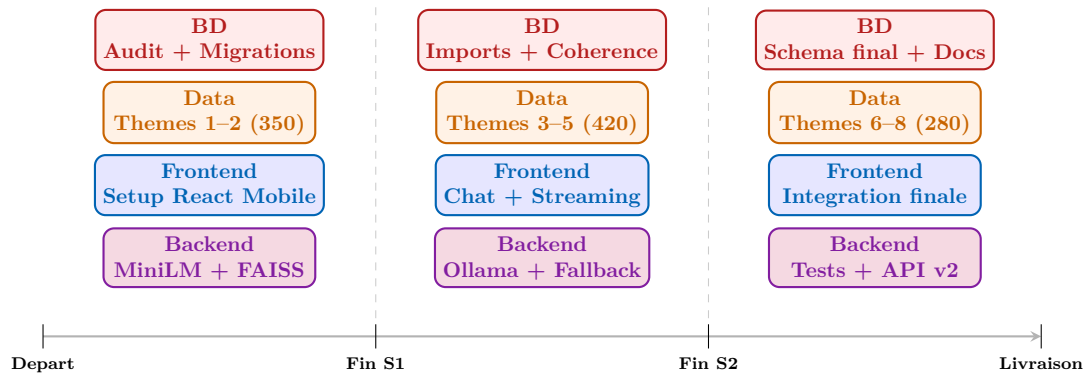


FIGURE 1 – Planning macroscopique — 3 semaines

4 Semaine 1 — Fondations (J1 a J5)

Objectifs de la semaine

- **[BACK]** MiniLM operationnel, index FAISS initial construit, LLM Phi-3 installé et testé
- **[FRONT]** Projet React créé (mobile-first), composants Header, ChatBubble, InputBar et FeedbackBar implémentés
- **[DATA]** 350 Q/R produits et validés (thèmes 1 et 2 + début 3), `verify_quota.py` opérationnel des J1
- **[BD]** Audit du schéma Phase 1 terminé, migrations préparées et testées, premiers imports

4.1 Jour 1 — Lundi : Lancement et configuration

Backend — J1

- ☐ Créer la structure `backend/chatbot/engine/` avec les fichiers vides suivants : `embedder.py`, `faiss_search.py`, `llm_client.py`, `prompt_builder.py`, `rag_pipeline.py`, `tfidf_fallback.py`
- ☐ Installer les dépendances : `pip install sentence-transformers faiss-cpu ollama`
- ☐ Mettre à jour `requirements.txt` et commiter sur la branche `backend`
- ☐ Vérifier que le modèle MiniLM (`all-MiniLM-L6-v2`) se charge correctement en local
- ☐ Installer Ollama et télécharger le modèle : `ollama pull phi3:mini`
- ☐ Vérifier qu'Ollama répond à une question test : `ollama run phi3:mini "Bonjour"`
- ☐ (Optionnel) Committer un `Dockerfile` de développement pour sécuriser l'environnement

Frontend — J1

- ☐ Créer le projet React : `npm create vite@latest supptic-chat --template react`
- ☐ Installer les dépendances : `npm install react-router-dom axios`
- ☐ Définir la structure des dossiers : `components/`, `screens/`, `services/`, `hooks/`, `assets/`
- ☐ Configurer les variables CSS globales dans `index.css` : couleur primaire bleu `#00468C`, blanc, gris clair, police système
- ☐ Configurer le viewport mobile dans `index.html` : `<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0">`
- ☐ Configurer Vite : port 3000, proxy vers Django sur le port 8000 (éviter les CORS en développement)
- ☐ Commit initial sur branche `frontend`

Data — J1

- ☐ Reunion de lancement : presentation des roles des binomes, lecture du guide JSON v2, rappel des quotas et du processus journalier
- ☐ Creer et tester `verify_quota.py` (cf. section ??) sur un fichier exemple factice
- ☐ **Binome A collecte (9h–12h)** : Theme 1 — Admissions et inscriptions. Chaque membre choisit un sous-theme distinct. Total attendu : **70 Q/R bruts**. Remise dans brouillons/ a 12h.
- ☐ **Binome B valide (13h–17h)** : lit et valide la production du Binome A. Corrige le format, note les rejets dans `brouillons/rejets_j1.json`.
- ☐ Commit soir par Binome B : `faq_admissions_j1.json`
- ☐ Lancer `verify_quota.py -file faq_admissions_j1.json -quota 70` : verifier que le quota est atteint

Base de Donnees — J1

- ☐ Audit complet des modeles Phase 1 : FAQ, Category, Feedback
- ☐ Identifier les champs manquants pour le RAG : `source` (provenance de la reponse), `embedding_id` (reference dans l'index FAISS), `rag_method` (methode ayant retourne la reponse)
- ☐ Rediger `schema_phase2.md` : schema ER actualise avec les nouveaux champs et leur type Django

4.2 Jour 2 — Mardi : Premiers embeddings et composants React**Backend — J2**

- ☐ Ecrire `embedder.py` : charger `all-MiniLM-L6-v2` une seule fois au demarrage, exposer `encode(text: str) -> np.ndarray` (vecteur de dimension 384)
- ☐ Ecrire `scripts/build_index.py` : lire les fichiers JSON v2, encoder chaque entree via `embedder.encode()`, generer `index.bin` (index FAISS) + `metadata.json` (mapping `id` ↔ reponse)
- ☐ Tester sur un echantillon de 50 Q/R de Phase 1 : verifier que les vecteurs sont generes sans erreur
- ☐ Benchmark : mesurer le temps de vectorisation d'une phrase (objectif < 50 ms)

Frontend — J2

- ☐ Creer **Header** : barre bleue (#00468C), icone menu hamburger (≡) a gauche, titre "Supone AI" centre, icone profil a droite. Hauteur fixe 56px, **position: sticky; top: 0; z-index: 100.**
- ☐ Creer **ChatBubble** : props `message` (texte), `isUser` (boolean), `timestamp`. Bulle utilisateur alignee a droite (fond bleu, texte blanc), bulle bot alignee a gauche (fond gris clair, texte noir). Border-radius 18px, padding 12px 16px, largeur max 80% de l'ecran.
- ☐ Creer **TypingIndicator** : trois points animes (CSS `@keyframes bounce`) affiches pendant la generation de la reponse.
- ☐ Tester l'affichage de plusieurs ChatBubble avec des donnees statiques dans `App.jsx`

Data — J2

- ☐ **Binome B collecte (9h–12h)** : Theme 1 — Admissions (sous-themes differents de J1). Total attendu : **70 Q/R bruts**. Remise dans brouillons/ a 12h.
- ☐ **Binome A valide (13h–17h)** : lit et valide la production du Binome B.
- ☐ Commit soir par Binome A : `faq_admissions_j2.json`
- ☐ Lancer `verify_quota.py` | Cumul : **140 Q/R valides**

Base de Donnees — J2

- ☐ Ecrire la premiere migration Django : ajout du champ `source` (CharField, `max_length=200`, `blank=True`) sur le modele FAQ
- ☐ Appliquer la migration : `python manage.py makemigrations && python manage.py migrate`
- ☐ Verifier que l'API Phase 1 (TF-IDF) fonctionne toujours apres migration (aucune regression)

4.3 Jour 3 — Mercredi : FAISS operationnel et layout chat**Backend — J3**

- ☐ Ecrire `faiss_search.py` : charger `index.bin` au demarrage, exposer `search(query_vec: np.ndarray, k: int = 3) -> list[tuple[int, float]]` (retourne des paires (`id_metadata`, `score`))
- ☐ Construire l'index initial avec toutes les donnees Phase 1 existantes via `build_index.py`
- ☐ Tester manuellement : 10 questions variees, verifier la pertinence semantique des top-3
- ☐ Assurer le chargement de l'index au demarrage de Django via `AppConfig.ready()`

Frontend — J3

- ☐ Creer **ChatScreen** : conteneur `overflow-y: auto`, auto-scroll vers le bas a chaque nouveau message (`useEffect` + `ref.scrollToView()`), padding bottom 80px pour ne pas etre cache par InputBar
- ☐ Creer **InputBar** : fixee en bas (`position: fixed; bottom: 0`), champ texte multi-lignes (max 4 lignes, auto-resize), bouton envoi bleu rond, gestion de la touche Entree (envoyer) vs Maj+Entree (saut de ligne). Hauteur minimale 56px.
- ☐ Assembler Header + ChatScreen + InputBar dans `ChatLayout.jsx`
- ☐ Tester l'ajout dynamique de bulles via React state : simuler une conversation de 5 echanges
- ☐ Verifier le rendu sur viewport 390px (iPhone 14) et 360px (Galaxy A) via Chrome DevTools

Data — J3

- ☐ **Binome A collecte (9h–12h)** : Theme 2 — Frais de scolarite, chaque membre choisit un sous-theme distinct. Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome B valide (13h–17h)** : valide la production du Binome A. Fusionner `faq_admissions_j1.json` + `faq_admissions_j2.json` en `faq_admissions.json` definitif.
- ☐ Commit soir par Binome B : `faq_frais_j3.json` + `faq_admissions.json` (complet)
- ☐ Lancer `verify_quota.py` | Cumul : **210 Q/R valides**

Base de Donnees — J3

- ☐ Ecrire `scripts/import_json.py` : lire un fichier JSON v2, inserer chaque entree en base, ignorer les doublons par verification de l'identifiant unique `id`
- ☐ Tester l'import de `faq_admissions.json` (140 entrees)
- ☐ Verifier l'absence de doublons apres import

4.4 Jour 4 — Jeudi : LLM Phi-3 et service API frontend**Backend — J4**

- ☐ Ecrire `llm_client.py` : connexion HTTP a Ollama (`localhost:11434`), exposer `generate(prompt: str) -> str` (reponse complete) et `generate_stream(prompt: str) -> Iterator[str]` (tokens en continu)
- ☐ Tester une generation manuelle : envoyer une question simple, verifier la reponse en francais
- ☐ Mesurer la latence de generation (objectif < 3s pour une reponse courte)
- ☐ Documenter les parametres Ollama utilises : `temperature`, `num_predict`, `stop`

Frontend — J4

- ☐ Créer `services/api.js` : fonction `askQuestion(text, history)` effectuant un POST `/api/chatbot/ask/`
- ☐ Gerer les etats dans le service : `loading`, `error`, `data`
- ☐ Afficher `TypingIndicator` pendant la requete API
- ☐ Tester la connexion avec l'API Phase 1 (TF-IDF) pour valider la plomberie reseau (CORS, format JSON)
- ☐ Créer **ErrorBanner** : bandeau rouge affichant les erreurs reseau (timeout, 500, etc.) avec un bouton "Reessayer"

Data — J4

- ☐ **Binome B collecte (9h–12h)** : Theme 2 — Frais de scolarite (sous-themes differents de J3). Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome A valide (13h–17h)** : valide la production du Binome B. Fusionner en `faq_frais.json` definitif.
- ☐ Commit soir par Binome A : `faq_frais.json` (140 Q/R complet)
- ☐ Lancer `verify_quota.py` | Cumul : **280 Q/R valides**

Base de Donnees — J4

- ☐ Import de `faq_frais.json` en base via `import_json.py`
- ☐ Verification croisee : nombre d'entrees en base vs. nombre d'entrees dans le JSON
- ☐ Rapport de coherence J4 : total Q/R en base, repartition par theme, champs nuls

4.5 Jour 5 — Vendredi : Bilan S1 et Pull Requests**Backend — J5**

- ☐ Ecrire `prompt_builder.py` : construire le prompt Phi-3 depuis les contextes FAISS (top-3) + question utilisateur. Message systeme : "Tu es un assistant de SUP'PTIC. Reponds uniquement en francais et en te basant sur le contexte fourni."
- ☐ Test end-to-end manuel : question → embed → FAISS → prompt → LLM → reponse affichee
- ☐ Verifier que le LLM reste dans le contexte fourni (ne fabrique pas d'informations non presentes)
- ☐ Pull Request **backend** → dev : MiniLM + FAISS + LLM + `prompt_builder` fonctionnels

Frontend — J5

- ☐ Creer **FeedbackBar** : barre d'actions sous chaque bulle bot contenant les boutons ✓ (like), ✗ (dislike), icone "Copier le texte", icone "Recharger la reponse"
- ☐ Etat des boutons : desactiver apres un premier clic, feedback visuel (✓ vert, ✗ rouge)
- ☐ Verifier l'affichage sur petits ecrans (360px) : la barre ne deborde pas
- ☐ Pull Request **frontend** → **dev** : Header, ChatBubble, InputBar, FeedbackBar, TypingIndicator, ErrorBanner

Data — J5

- ☐ **Binome A collecte (9h–12h)** : Theme 3 — Logement (debut). Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome B valide (13h–17h)** : valide la production du Binome A + correction retroactive des rejets des jours precedents
- ☐ Bilan S1 Data : qualite generale, taux de rejet, points a ameliorer
- ☐ Pull Request **data** → **dev** : `faq_admissions.json` + `faq_frais.json`
- ☐ Lancer `verify_quota.py` | Cumul : $\geq$ **350 Q/R valides**

Base de Donnees — J5

- ☐ Declencher `build_index.py` sur toutes les donnees importees (S1 complete)
- ☐ Verifier que l'index FAISS charge par Django est coherent avec la base
- ☐ Pull Request **database** → **dev** : migrations + `import_json.py`

Bilan Semaine 1 — Criteres de succes

- ✓ MiniLM charge et vectorise correctement (test sur 50 Q/R)
- ✓ Index FAISS construit et interrogeable (top-3 pertinent sur 10 questions test)
- ✓ LLM Phi-3 installe, repond en moins de 3s
- ✓ Composants Header + ChatBubble + InputBar + FeedbackBar valides (mobile 390px et 360px)
- ✓ 350 Q/R valides (themes 1 et 2 + debut 3), `verify_quota.py` operationnel
- ✓ Migrations BD appliquees (champ `source`), import fonctionne

5 Semaine 2 — Le Coeur Intelligent (J6 a J10)

Objectifs de la semaine

- **[BACK]** Pipeline RAG complet, endpoint `/api/chatbot/ask/` v2 avec streaming SSE et fallback TF-IDF
- **[FRONT]** Ecran de chat complet : streaming visible, sources, badge methode, historique, feedback connecte
- **[DATA]** +420 Q/R (themes 3, 4, 5), imports quotidiens en base
- **[BD]** Imports batch apres chaque journee de validation, audit de coherence mi-parcours

5.1 Jour 6 — Lundi : Pipeline RAG et streaming

Backend — J6

- ☐ Ecrire `rag_pipeline.py` : orchestrer `embed` → FAISS (top-3) → prompt → LLM
- ☐ Exposer dans `chatbot/views.py` : POST `/api/chatbot/ask/` v2
- ☐ Format de la reponse JSON : `{"answer": "...", "sources": [{"question": "...", "score": 0.82}], "method": "RAG"}`
- ☐ Activer le streaming SSE : `StreamingHttpResponse` Django avec `Content-Type: text/event-stream`, chaque token du LLM est envoye comme un evenement `data: <token>\n\n`
- ☐ Tester avec curl : `curl -N -X POST /api/chatbot/ask/ -d '{"question": "Quels sont les frais?"}'`

Frontend — J6

- ☐ Implementer le **streaming** : utiliser l'API `fetch` avec `ReadableStream` pour lire le flux SSE octet par octet. A chaque chunk reçu, concatener le texte dans la bulle en cours de construction.
- ☐ Creer le hook `useChat()` : gere la liste des messages, l'etat `isStreaming`, l'accumulation du texte streamé et l'ajout de la bulle finale une fois le stream termine
- ☐ Mettre a jour `ChatBubble` pour accepter un mode "streaming" : afficher le texte progressivement
- ☐ Afficher `TypingIndicator` uniquement quand `isStreaming === true` et que la bulle est encore vide
- ☐ Tester visuellement l'effet de frappe progressive sur viewport mobile

Data — J6

- ☐ **Binome B collecte (9h–12h)** : Theme 3 — Logement (suite, sous-themes differents de J5). Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome A valide (13h–17h)** : valide la production du Binome B. Fusionner avec la production J5 pour obtenir `faq_logement.json` complet (140 Q/R).
- ☐ Commit soir par Binome A : `faq_logement.json`
- ☐ Lancer `verify_quota.py` | Cumul : **490 Q/R valides**

Base de Donnees — J6

- ☐ Import `faq_logement.json` via `import_json.py`
- ☐ Declencher le rebuild FAISS, verifier que Django charge le nouvel index sans redemarrage (via `reload_trigger.py`)
- ☐ Verifier la coherence relationnelle : toutes les entrees ont une categorie assignee

5.2 Jour 7 — Mardi : Fallback TF-IDF et affichage des sources**Backend — J7**

- ☐ Ecrire `tfidf_fallback.py` : encapsuler le moteur TF-IDF de Phase 1, exposer `search(query: str) -> tuple[str, float, str]` (reponse, score, methode="TF-IDF")
- ☐ Integrer la bascule dans `rag_pipeline.py` : si le meilleur score FAISS < 0.55 , appeler `tfidf_fallback.search()`
- ☐ Le champ `method` de la reponse finale vaut "RAG" ou "TF-IDF" selon la branche empruntee
- ☐ Tester avec des questions hors domaine (ex : "Quel temps fait-il ?") : la bascule TF-IDF doit s'activer
- ☐ Tester avec des questions bien formulees sur SUP'PTIC : la voie RAG doit etre privilegiee
- ☐ Ajuster le seuil si necessaire (valeur par default : 0.55)

Frontend — J7

- ☐ Creer **SourcesCard** : composant affichant les sources retournees par l'API. Chaque carte montre la question source et le score de confiance (arrondi en pourcentage). Composant pliable/depliable, ferme par default.
- ☐ Creer **MethodBadge** : badge colore affiche dans la bulle bot. Valeur RAG en violet, valeur TF-IDF en orange.
- ☐ Integrer SourcesCard et MethodBadge sous chaque bulle bot dans ChatScreen
- ☐ Tester visuellement : la carte sources s'ouvre et se ferme au tap, l'animation est fluide

Data — J7

- ☐ **Binome A collecte (9h–12h)** : Theme 4 — Emploi du temps, chaque membre choisit un sous-theme distinct. Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome B valide (13h–17h)** : valide la production du Binome A. Commit `faq_emploi_j7.json`.
- ☐ Lancer `verify_quota.py` | Cumul : **560 Q/R valides**

Base de Donnees — J7

- ☐ Import `faq_emploi_j7.json`
- ☐ Verification des doublons inter-themes
- ☐ Rapport de coherence : champs manquants, valeurs nulles, FK orphelines

5.3 Jour 8 — Mercredi : Historique et memoire contextuelle

Backend — J8

- ☐ Modifier `prompt_builder.py` : accepter un parametre `history: list[dict]` (liste des 3 derniers echanges) et l'inclure dans le prompt envoye au LLM. Format : `[Historique]\nQ: ... \nR: ... \n... \n[Nouvelle question]\n{question}`
- ☐ Modifier l'endpoint `/api/chatbot/ask/` pour accepter le champ `history` dans le corps de la requete (peut etre absent ou vide pour la premiere question)
- ☐ Tester une conversation multi-tours : “Quels sont les frais?” puis “Et pour les boursiers?” — la deuxieme reponse doit tenir compte du contexte

Frontend — J8

- ☐ Creer **HistorySidebar** : panneau lateral listant les conversations passees (titre automatique = premiere question posee, date)
- ☐ Declencher l'ouverture via le bouton `≡` du Header, animation `slide-in` depuis la gauche (`transform: translateX` avec transition 300ms)
- ☐ Stocker les conversations en `localStorage` : cle `supptic_conversations`, tableau d'objets `{id, title, messages, date}`
- ☐ Permettre de reprendre une conversationensee en cliquant dessus : charger les messages dans `ChatScreen`
- ☐ Bouton “Nouvelle conversation” en haut du sidebar : vider l'etat courant, generer un nouvel id
- ☐ Envoyer les 3 derniers messages comme `history` dans chaque requete API via `useChat()`

Data — J8

- ☐ **Binome B collecte (9h–12h)** : Theme 4 — Emploi du temps (suite). Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome A valide (13h–17h)** : valide la production du Binome B. Fusionner les deux journees en `faq_emploi_du_temps.json` definitif (140 Q/R).
- ☐ Commit soir par Binome A : `faq_emploi_du_temps.json`
- ☐ Lancer `verify_quota.py` | Cumul : **630 Q/R valides**

Base de Donnees — J8

- ☐ Import `faq_emploi_du_temps.json` via `import_json.py`
- ☐ Ecrire `scripts/reload_trigger.py` : signaler au backend de recharger l'index FAISS a chaud sans redemarrer Django (par exemple via un fichier sentinelle ou un signal interne)
- ☐ Tester le rechargement a chaud : ajouter une entree en base, declencher le reload, verifier qu'elle est retrouvable par FAISS

5.4 Jour 9 — Jeudi : Feedback connecte et tests unitaires

Backend — J9

- ☐ Ajouter le champ `rag_method` au modele `Feedback` pour tracer quelle methode (RAG ou TF-IDF) a produit la reponse evaluee
- ☐ Ecrire `tests/test_embedder.py` : verifier que deux questions synonymes ont une similarite cosinus > 0.85
- ☐ Ecrire `tests/test_faiss_search.py` : verifier que les top-3 retournes sont semantiquement coherents sur 20 questions connues
- ☐ Ecrire `tests/test_fallback.py` : verifier que la bascule TF-IDF s'active bien pour les questions hors domaine (score < 0.55)
- ☐ Lancer `python manage.py test` et corriger les echecs

Frontend — J9

- ☐ Connecter `FeedbackBar` a l'endpoint `POST /api/feedback/` : envoyer `{message_id, type: "like" | "dislike"}`
- ☐ Retour visuel apres feedback : bouton colore (✓ vert, ✗ rouge), desactivation du double-clic
- ☐ Creer **SplashScreen** : ecran d'accueil affiche 2 secondes au lancement. Logo Supone AI centre sur fond bleu #00468C, animation de fondu (`opacity: 0 -> 1` en 500ms). Navigation automatique vers `ChatScreen` apres 2s via `useNavigate`.
- ☐ Configurer le routeur React : `/` → `SplashScreen`, `/chat` → `ChatLayout`, `/about` → `AboutScreen`
- ☐ Tester le flux complet : ouverture app → `SplashScreen` → Chat → question → reponse streamee → feedback

Data — J9

- ☐ **Binome A collecte (9h–12h)** : Theme 5 — Examens et notes, chaque membre choisit un sous-theme distinct. Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome B valide (13h–17h)** : valide la production du Binome A. Commit `faq_examens_j9.json`.
- ☐ Lancer `verify_quota.py` | Cumul : **700 Q/R valides**

Base de Donnees — J9

- ☐ Migration : ajout du champ `rag_method` (`CharField`, `max_length=10`, `default="TF-IDF"`) sur le modele `Feedback`
- ☐ Import `faq_examens_j9.json`
- ☐ Stats mi-parcours : total Q/R en base, repartition par theme, taux de champs remplis

5.5 Jour 10 — Vendredi : Bilan S2 et Pull Requests

Backend — J10

- ☐ Tests end-to-end : 20 questions variees couvrant les 5 themes importes, verifier bascule RAG/TF-IDF, champ `method`, streaming SSE
- ☐ Mesurer les temps de reponse : objectif moyenne $< 3s$ sur les 20 requetes. Documenter les resultats dans un fichier `perf_s2.md`.
- ☐ Optimiser si necessaire : reduire `num_predict` Ollama, limiter le contexte FAISS a 2 sources si la latence est elevee
- ☐ Pull Request `backend` → `dev` : pipeline RAG complet + fallback TF-IDF + tests unitaires

Frontend — J10

- ☐ Verifier le comportement mobile : gestion du clavier virtuel (viewport resize), scroll automatique, InputBar qui remonte avec le clavier
- ☐ Corriger tous les bugs UI identifies cette semaine
- ☐ Tester sur un appareil Android reel si possible (pas seulement Chrome DevTools)
- ☐ Pull Request `frontend` → `dev` : chat complet (streaming, sources, historique, feedback, SplashScreen, routeur)

Data — J10

- ☐ **Binome B collecte (9h–12h)** : Theme 5 — Examens (suite). Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome A valide (13h–17h)** : valide la production du Binome B. Fusionner en `faq_examens.json` definitif (140 Q/R).
- ☐ Commit soir par Binome A : `faq_examens.json`
- ☐ Bilan S2 : lancer `verify_quota.py` | Cumul : $\geq$ **770 Q/R valides**
- ☐ Pull Request `data` → `dev` : themes 3 + 4 + 5 complets

Base de Donnees — J10

- ☐ Import `faq_examens.json`
- ☐ Rebuild complet de l'index FAISS avec toutes les donnees S1 + S2
- ☐ Audit de coherence global : relations, doublons, champs nuls, distribution par theme
- ☐ Pull Request `database` → `dev`

Bilan Semaine 2 — Criteres de succes

- ✓ Pipeline RAG complet, expose via `/api/chatbot/ask/` v2 avec SSE
- ✓ Fallback TF-IDF operationnel, bascule automatique validee
- ✓ Streaming visible dans l'interface React
- ✓ Sources et badge methode affichees
- ✓ Historique des conversations fonctionne (localStorage)
- ✓ Feedback connecte a l'API backend
- ✓ SplashScreen et routeur React en place
- ✓ 770 Q/R valides en base (themes 1 a 5)

6 Semaine 3 — Finalisation et Demo (J11 a J15)

Objectifs de la semaine

- **[BACK]** API stable et documentee, tests de charge reussis, documentation complete
- **[FRONT]** Application React mobile entierement integree avec le backend RAG, prete pour la demo
- **[DATA]** +280 Q/R (themes 6, 7, 8), dataset final livre pour l'index definitif
- **[BD]** Schema final valide, index FAISS synchronise avec toutes les donnees, documentation livree

6.1 Jour 11 — Lundi : Integration globale

Backend — J11

- ☐ Tester le backend avec le frontend React : verifier les CORS, les formats de reponse JSON, les codes d'erreur HTTP
- ☐ Corriger les incompatibilites de format entre les deux equipes (noms de champs, casse, pagination)
- ☐ Ajouter un cache memoire simple (dictionnaire Python) pour les embeddings des questions les plus frequentes (optionnel, si la latence est elevee)
- ☐ Verifier que tous les tests unitaires passent toujours apres les corrections

Frontend — J11

- ☐ Basculer de l'API Phase 1 (TF-IDF) vers l'API Phase 2 (RAG) dans `services/api.js`
- ☐ Tester chaque composant avec la vraie reponse RAG : streaming, sources, badge methode, feedback
- ☐ Corriger les bugs d'integration identifies lors des tests croises
- ☐ Creer **AboutScreen** : ecran "A propos" accessible depuis le Header (icone info ou menu). Explique en langage simple ce qu'est le RAG, FAISS et le LLM. Bouton retour vers ChatScreen.

Data — J11

- ☐ **Binome A collecte (9h–12h)** : Theme 6 — Filieres et parcours, chaque membre choisit un sous-theme distinct. Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome B valide (13h–17h)** : valide la production du Binome A. Commit `faq_filieres_j11.json`.
- ☐ Lancer `verify_quota.py` | Cumul : **840 Q/R valides**

Base de Donnees — J11

- ☐ Import `faq_filieres_j11.json`, declencher rebuild FAISS partiel
- ☐ Verifier la coherence relationnelle : FK, categories bien assignees, absence de doublons

6.2 Jour 12 — Mardi : Polissage UI/UX et tests de charge

Backend — J12

- ☐ Test de charge : lancer 50 requetes consecutives, verifier la stabilite du serveur et la coherence des reponses
- ☐ Rediger `FLUX_RAG.md` : explication du pipeline etape par etape (embed → FAISS → score → bascule → LLM → SSE)
- ☐ Rediger `OLLAMA_SETUP.md` : guide d'installation, demarrage, parametres utilises
- ☐ Documenter tous les endpoints avec leur format exact (requete + reponse exemple + codes d'erreur)

Frontend — J12

- ☐ Animations d'apparition progressive des bulles : entree par le bas (`transform: translateY(20px) -> 0` en 200ms)
- ☐ Gestion des erreurs reseau dans `ErrorBanner` : afficher un message convivial avec bouton "Reessayer"
- ☐ Verifier le responsive sur plusieurs largeurs : 360px (Galaxy A), 390px (iPhone 14), 430px (iPhone 14 Plus)
- ☐ Tester sur navigateur mobile reel : comportement du clavier, scroll, tap, double-tap
- ☐ Creer **StatsScreen** (optionnel) : ecran statistiques affichant le nombre de questions posees, taux de satisfaction (feedbacks positifs / total), methode la plus utilisee (RAG ou TF-IDF)

Data — J12

- ☐ **Binome B collecte (9h–12h)** : Theme 6 — Filieres (suite). Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome A valide (13h–17h)** : valide la production du Binome B. Fusionner en `faq_filieres.json` definitif (140 Q/R).
- ☐ Commit soir par Binome A : `faq_filieres.json`
- ☐ Lancer `verify_quota.py` | Cumul : **910 Q/R valides**

Base de Donnees — J12

- ☐ Import `faq_filieres.json`
- ☐ Verifier les scores FAISS sur les nouvelles donnees (qualite des embeddings)
- ☐ Rediger `FAISS_SETUP.md` : construction de l'index, format de `metadata.json`, reload a chaud

6.3 Jour 13 — Mercredi : Documentation et avant-demo

Backend — J13

- ☐ Mettre a jour le README principal (section Phase 2) : architecture, installation, démarrage
- ☐ Rediger CHANGELOG.md Phase 2 : liste de toutes les nouvelles fonctionnalites
- ☐ Verifier la documentation de chaque endpoint : `/api/chatbot/ask/`, `/api/feedback/`, `/api/stats/`, `/api/reload-index/`

Frontend — J13

- ☐ Finaliser SplashScreen : logo centre, animation fluide, redirection automatique apres 2s
- ☐ Preparer le scenario de demo : 8 questions representatives (2 questions simples RAG, 2 questions de bascule TF-IDF, 2 questions de suivi de conversation multi-tours, 2 questions pour le feedback)
- ☐ Revue de code frontend : lisibilite, suppression des `console.log`, nettoyage des imports inutilises
- ☐ Pull Request finale `frontend` → `dev`

Data — J13

- ☐ **Binome A collecte (9h–12h)** : Theme 7 — Services administratifs, chaque membre choisit un sous-theme distinct. Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **Binome B valide (13h–17h)** : valide la production du Binome A. Commit `faq_services_admin_j13.json`.
- ☐ Lancer `verify_quota.py` | Cumul : **980 Q/R valides**

Base de Donnees — J13

- ☐ Import `faq_services_admin_j13.json`
- ☐ Audit final du schema relationnel : tous les champs ajoutes en Phase 2 sont renseignes
- ☐ Rediger `GUIDE_FORMAT_JSON.md` : guide complet pour l'equipe Data (format, regles, exemples)
- ☐ Pull Request `database` → `dev`

6.4 Jour 14 — Jeudi : Repetition generale et clotûre Data

Toutes les equipes convergent pour les tests finaux et la preparation de la demo.

Matin — Tests croises (9h–12h)

Fonctionnalite	Backend	Frontend	Data
Embedding + FAISS (top-3 pertinent)	✓		
LLM streaming SSE	✓	✓	
Fallback TF-IDF (bascule auto)	✓	✓	
Affichage sources + badge methode		✓	
Feedback connecte	✓	✓	
Historique conversations (localStorage)		✓	
Memoire conversationnelle (multi-tours)	✓	✓	
Indexation des nouvelles donnees	✓		✓

Apres-midi — Preparation demo (13h–17h)

- ☐ **[BACK]** Rebuild index FAISS final avec le dataset complet
- ☐ **[BACK]** Tag `v2.0.0-rc` sur la branche `dev`
- ☐ **[FRONT]** Tester le scenario de demo de bout en bout (les 8 questions preparees en J13)
- ☐ **[FRONT]** Verifier streaming, sources, fallback, feedback, historique sur l'application finale
- ☐ **[DATA]** **Binome B collecte (matin)** : Theme 7 suite + debut Theme 8 — Vie etudiante. Total attendu : **70 Q/R bruts**. Remise a 12h.
- ☐ **[DATA]** **Binome A valide (apres-midi)** : valide et commit `faq_services_admin.json` complet (140 Q/R)
- ☐ **[BD]** Import `faq_services_admin.json` | Cumul : ≥ 1050 Q/R
- ☐ **[BD]** Stats finales : total Q/R en base, repartition par theme, taux de champs remplis
- ☐ Repetition demo (simulation 30 min, toutes equipes presentes)
- ☐ Etablir la liste des ajustements a faire J15 matin

6.5 Jour 15 — Vendredi : Demo officielle et livraison**6.5.1 Matin — Derniers réglages (9h–13h)**

- ☐ Corrections issues de la liste J14
- ☐ **[DATA]** Commit `faq_vie_etudiante.json` (Theme 8) | Cumul final : ≥ 1050 Q/R
- ☐ **[BD]** Import final + rebuild FAISS definitif sur le dataset complet
- ☐ Merge `dev` → `main`, tag `v2.0.0`

6.5.2 Après-midi — Demo officielle (14h–17h)

Deroulement de la demo (45 min)

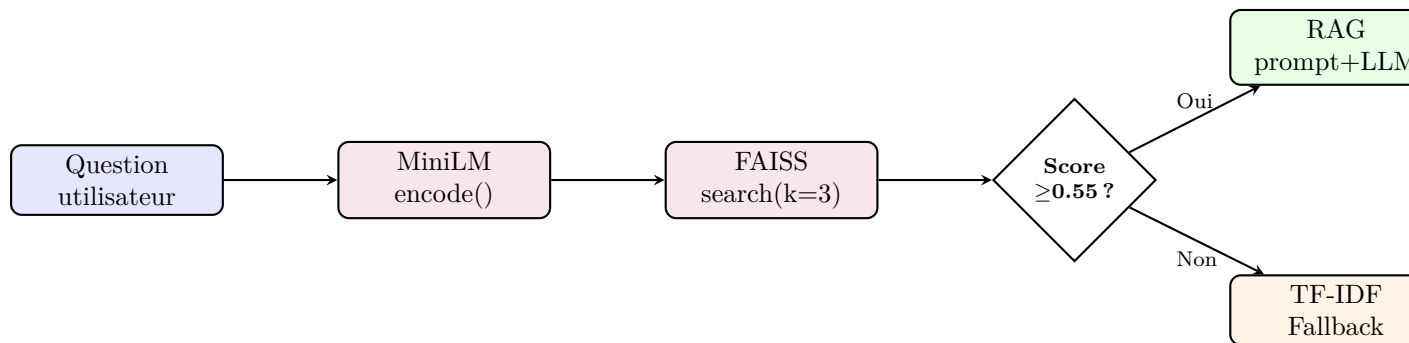
1. **Contexte et objectifs Phase 2** (5 min) : rappel des limites Phase 1, solutions adoptees, equipes impliquees
2. **Architecture RAG** (5 min) : schema question → MiniLM → FAISS → bascule → Phi-3 → reponse streamee
3. **Demonstration live** (25 min) :
 - Question bien formulee → reponse RAG, streaming visible, badge RAG, sources affichees
 - Question mal formulee ou hors domaine → bascule TF-IDF, badge TF-IDF
 - Suivi de conversation multi-tours (memoire contextuelle)
 - Envoi d'un feedback (like / dislike) et confirmation visuelle
 - Ouverture de HistorySidebar et reprise d'une conversation passee
4. **Dataset** (5 min) : 1050+ Q/R, 8 themes couverts, presentation du processus binomes et de `verify_quota.py`
5. **Questions et retrospective** (5 min)

6.5.3 Soir — Livraison finale

- ☐ Tag `v2.0.0` sur `main`, creation de la release GitHub avec notes de version
- ☐ Archive des branches `backend`, `frontend`, `data`, `database`

7 Architecture technique

7.1 Pipeline RAG



7.2 Structure des fichiers backend

Listing 3 – Arborescence backend (engine)

```

backend/chatbot/
  engine/
    embedder.py          # Chargement MiniLM, encode()
    faiss_search.py      # Index FAISS, search()
    llm_client.py        # Client Ollama, generate() et generate_stream()
    prompt_builder.py    # Construction prompt (contexte + historique)
    rag_pipeline.py      # Orchestrateur principal
    tfidf_fallback.py    # Encapsulation moteur TF-IDF Phase 1
  views.py               # Endpoints Django
  tests/
    test_embedder.py
    test_faiss_search.py
    test_fallback.py
  scripts/
    build_index.py       # Genere index.bin + metadata.json
    reload_trigger.py    # Reload FAISS a chaud
    import_json.py       # Import fichiers JSON v2 en base
    verify_quota.py      # Verification quotas Data
  
```

7.3 Structure des composants React

Listing 4 – Arborescence frontend

```

supptic-chat/src/
  screens/
    SplashScreen.jsx     # Ecran d'accueil (2s, logo anime, fond bleu)
    ChatScreen.jsx       # Ecran principal de chat (layout complet)
    AboutScreen.jsx      # Presentation du projet RAG
  components/
    Header.jsx           # Barre de navigation superieure (sticky)
    ChatBubble.jsx       # Bulle de message (user/bot + streaming)
    InputBar.jsx          # Zone de saisie fixee en bas
    FeedbackBar.jsx       # Boutons like/dislike/copier/recharger
    SourcesCard.jsx       # Cartes sources pliablles
    MethodBadge.jsx       # Badge RAG (violet) ou TF-IDF (orange)
    TypingIndicator.jsx   # Animation "..." pendant la generation
  
```

HistorySidebar.jsx	# Panneau historique (slide-in)
ErrorBanner.jsx	# Bandeau d'erreur reseau
hooks/	
useChat.js	# Gestion messages + streaming SSE
services/	
api.js	# Appels API Django

8 Recapitulatif des livrables

Equipe	Livrable	Deadline
Backend	engine/ complet : embed- der, faiss_search, llm_client, prompt_builder, rag_pipeline, tfidf_fallback	J6
Backend	Endpoint /api/chatbot/ask/ v2 avec SSE et fallback	J7
Backend	Tests unitaires : test_embedder, test_faiss_search, test_fallback	J9
Backend	FLUX_RAG.md, OLLAMA_SETUP.md, CHANGELOG.md, README v2	J13
Frontend	SplashScreen, Header, Chat- Bubble, InputBar, FeedbackBar, TypingIndicator, ErrorBanner	J5
Frontend	SourcesCard, MethodBadge, hook useChat(), streaming SSE	J7
Frontend	HistorySidebar, memoire conver- sationnelle (localStorage), rou- teur	J8
Frontend	AboutScreen, integration com- plete avec backend RAG	J13
Data	verify_quota.py operationnel	J1
Data	faq_admissions.json + faq_frais.json (280 Q/R)	J4
Data	faq_logement.json + faq_emploi_du_temps.json + faq_examens.json (420 Q/R)	J10
Data	faq_filieres.json + faq_services_admin.json + faq_vie_etudiante.json (350 Q/R)	J14
Data	Total ≥ 1050 Q/R nouveaux valides	J15
BD	Migration champ source sur FAQ	J2
BD	import_json.py (import sans doublons)	J3
BD	Migration champ rag_method sur Feedback	J9
BD	reload_trigger.py (reload FAISS a chaud)	J8
BD	Index FAISS synchronise apres chaque import	continu
BD	FAISS_SETUP.md, GUIDE_FORMAT_JSON.md, schema_phase2.md	J13

9 Indicateurs de succes

Critere	Objectif	Comment mesurer
Q/R valides et en base	≥ 1050 nouveaux	<code>verify_quota.py</code> + stats BD
Taux de reponse RAG (score ≥ 0.55)	$> 60\%$	Logs API sur 50 questions variees
Temps de reponse RAG	< 3 s	Moyenne sur 20 requetes (<code>perf_s2.md</code>)
Composants React fonctionnels	100% de la liste	Revue de code
Integration frontend-backend	Aucune erreur CORS	Test sur navigateur mobile reel
Tests unitaires backend	100% passants	<code>python manage.py test</code>
Demo reussie	✓	Validation par l'equipe

10 Risques et plans de contingence

Risque	Prob.	Impact	Plan B
Ollama/Phi-3 trop lent sur la machine de demo	Moyenne	Eleve	Reduire le contexte FAISS a 1 source, abaisser <code>num_predict</code> , basculer sur TF-IDF seul le temps de la demo
Quota Data non atteint un jour	Moyenne	Modere	Le lendemain, le binome collecteur cible un sous-theme plus simple pour recuperer les Q/R manquants
Entrees JSON massive-ment rejetees	Faible	Modere	Script <code>auto_correct.py</code> qui tente de corriger le format automatiquement avant rejet definitif
React non maitrise par l'equipe	Moyenne	Eleve	Maintien de la PWA Phase 1 en parallele; les composants Chat et Input sont priorises sur les ecrans secondaires
FAISS index trop volumineux	Faible	Faible	Compression (<code>faiss.IndexIVFFlat</code>)
Probleme de dependances Python	Faible	Eleve	Dockerfile de developpement commite des J1
CORS Django bloquant le frontend	Faible	Modere	Verifier <code>CORS_ALLOWED_ORIGINS</code> en J4, proxy Vite en developpement

Synthese — Phase 2 en bref

3 semaines pour transformer le chatbot

Semaine	Theme	Livrable principal
S1	Fondations	MiniLM + FAISS + LLM installe + React setup + 350 Q/R
S2	Coeur intelligent	Pipeline RAG + Fallback TF-IDF + streaming + sources + historique + 420 Q/R
S3	Finalisation	Integration complete + tests + documentation + demo + 280 Q/R

Ce qui est nouveau par rapport a la Phase 1

- ✓ Recherche semantique (MiniLM) au lieu de TF-IDF seul
- ✓ Index vectoriel FAISS (`index.bin` + `metadata.json`)
- ✓ Generation de reponse naturelle par LLM (Phi-3 / Ollama)
- ✓ Bascule automatique RAG ↔ TF-IDF selon le score de confiance
- ✓ Interface React mobile (remplacement de la PWA)
- ✓ Streaming des reponses visible dans l'interface
- ✓ Affichage des sources et scores de confiance
- ✓ Historique des conversations (`localStorage`)
- ✓ Memoire conversationnelle (3 derniers echanges dans le prompt)
- ✓ 1050+ nouveaux Q/R produits par le processus de binomes circulaires
- ✓ Script `verify_quota.py` garantissant les quotas quotidiens

Document prepare par :
NKOUMOU TJADE
 Chef de la Division de Programmation
 Club Informatique SUP'PTIC

Bonne Phase 2 a toutes les equipes !